

AkiNavi HR-AI 環境構築ガイド

このガイドに沿って作業することで、**メール自動取り込みを含むシステム全体**を新しい環境でゼロから動かすことができます。

作業時間の目安: 3〜6時間（初回）

全体の流れ

第0章: Outlookアカウントを4つ作る

↓

第1章: 必要なツールの準備・ソースコードへのアクセス

↓

第2章: データベースの作成 (Supabase)

↓

第3章: AIの設定 (Gemini APIキーの取得)

↓

第4章: サーバー機能のデプロイ (Edge Functions)

↓

第5章: メール自動取り込みの設定 (Azure + Microsoft Graph API)

↓

第6章: 本番サイトの公開 (Vercel) ← ここで環境変数をまとめて設定して完成

ローカル環境（自分のPCでの開発サーバー起動）は使いません。
設定はすべて Vercel と Supabase のダッシュボード上で行います。

第0章 Outlookメールアドレスを4つ作る

なぜ4つ必要か

このシステムは、専用のメールアドレスに転送・送信されたメールを自動で解析してデータベースに保存します。
用途ごとに別のメールアドレスを使うため、合計4つが必要です。

用途	例
人材情報の受信（本番用）	yourname.hr.human@outlook.jp
案件情報の受信（本番用）	yourname.hr.project@outlook.jp
人材情報の受信（デモ用）	yourname.hr.human.dev@outlook.jp
案件情報の受信（デモ用）	yourname.hr.project.dev@outlook.jp

yourname の部分は自分で決めた名前や識別子に置き換えてください。
例：会社名・サービス名・自分のイニシャルなど。他の人がすでに使っているアドレスは取得できないため、ユニークになるよう工夫してください。
例：**akinavi.hr.human@outlook.jp**、**tanaka2024.hr.human@outlook.jp** など

すべて同じ Microsoft アカウントではなく、それぞれ別のアカウントとして作成してください。 1アカウント = 1メールアドレスです。

なぜ Outlook (Microsoft) でないといけないのか

このシステムのメール自動取り込みは、**Microsoft Graph API** という仕組みを使ってメールを取得しています。Microsoft Graph API は **Microsoft のメールサービス (Outlook / Hotmail / Live) 専用** のAPIです。

メールサービス	使えるか	理由
Outlook / Hotmail / Live (@outlook.jp / @outlook.com など)	使える	Microsoft Graph API に対応している
Gmail (@gmail.com)	使えない	Google のサービスのため Graph API が使えない
Yahoo!メール	使えない	同上
会社の独自ドメインメール	条件次第	Microsoft 365 (旧Office 365) のメールであれば対応可能だが設定が複雑になる

Gmail などを使いたい場合は、このシステムの `poll-email` Edge Function を Gmail の API (Google Workspace API) 向けに作り直す必要があります。現状は対応していません。

作成手順 (1アカウントあたり)

1. Microsoftアカウント作成ページを開く

ブラウザで <https://outlook.com> を開き、「無料アカウントを作成」をクリック。

2. メールアドレスを決める

「新しいメールアドレスを取得」を選び、希望のメールアドレスを入力して「次へ」。

3. パスワード・名前・生年月日などを設定して完了

4. 同じ手順を繰り返す

上記を 4回繰り返し、4つのアカウントを作成する。

重要: 4つのメールアドレスと、それぞれのパスワードをメモしておいてください。後の章で使います。

完了チェック

- ☐ Outlookアカウントを4つ作成した
- ☐ 4つのメールアドレスとパスワードをメモした

第1章 必要なツールの準備・ソースコードへのアクセス

1-1. 必要なツールをインストールする

以下の2つをインストールします。

Git (ソースコードをダウンロードするためのツール)

ターミナルを開いて以下を入力し、バージョン番号が表示されればインストール済みです。

```
git --version
```

表示されない場合は <https://git-scm.com> からインストールしてください。

Supabase CLI（サーバー機能をデプロイするためのツール）

Mac の場合:

```
brew install supabase/tap/supabase
```

確認:

```
supabase --version
```

バージョンが表示されればOKです。

1-2. GitHubアカウントを作成する

「GitHub」とは、プログラムのソースコードを保管・管理するためのサービスです。ソースコードをダウンロードするために必要です。

1. <https://github.com> を開いて「Sign up」からアカウントを作成する

すでにアカウントがある場合はこの手順はスキップしてください。

2. 作成したGitHubアカウントのユーザー名をメモする

後の手順で担当者に伝える必要があります。

1-3. ソースコードへのアクセス権を担当者に依頼する

重要: このシステムのソースコード（[akinavi-hr-ai](#)）は非公開（Private）リポジトリです。
担当者に許可してもらわないと、ソースコードをダウンロードすることができません。

担当者に以下を伝えてください:

```
GitHubのユーザー名: （1-2でメモしたユーザー名）  
理由: akinavi-hr-ai のソースコードへのアクセス権（Collaborator）を付与してほしい
```

担当者が GitHub のリポジトリ設定からあなたを招待します。
招待メールが届いたら「Accept invitation」をクリックして承諾してください。

アクセス権が付与されるまでこの先の手順は進められません。担当者の対応を待ってください。

1-4. ソースコードをダウンロードする

アクセス権が付与されたら、ソースコードをダウンロードします。

```
git clone https://github.com/kzmiyamura/akinavi-hr-ai.git
cd akinavi-hr-ai
```

完了チェック

- ☐ Git をインストールした
- ☐ Supabase CLI をインストールした
- ☐ GitHubアカウントを作成し、ユーザー名をメモした
- ☐ 担当者にアクセス権を依頼し、招待を承諾した
- ☐ `git clone` でソースコードをダウンロードした

第2章 データベースの作成（Supabase）

「データベース」とは、人材情報・案件情報などのデータを保存する場所です。このシステムでは **Supabase**（無料で使えるデータベースサービス）を使います。

2-1. Supabaseにサインアップ・プロジェクトを作成する

1. <https://supabase.com> を開いてアカウントを作成する

「Start your project」をクリック → GitHubアカウントでサインアップすると簡単です。

2. 新しいプロジェクトを作成する

ダッシュボードの「New project」をクリックし、以下を入力して「Create new project」。

- **Name:** 任意（例: `akinavi-hr-ai`）
- **Database Password:** 自分でパスワードを設定（メモしておく）
- **Region:** `Northeast Asia (Tokyo)` を選択

プロジェクトの作成が完了するまで1〜2分待ちます。

3. 接続情報をメモする

プロジェクトが作成されたら、左メニューの「Settings」→「API」を開く。

以下の3つをメモしてください（第6章のVercel設定で使います）：

メモするもの	場所	用途
Project URL	「Project URL」の欄	アプリからDBへの接続先
anon（公開）キー	「Project API keys」の「anon」の欄	アプリがDBに接続するための鍵
service_role キー	「Project API keys」の「service_role」の欄	サーバー機能がDBを操作するための鍵（絶対に他人に見せない）

2-2. データベースのテーブルを作成する

「テーブル」とは、データベースの中の表（Excel のシートのようなもの）です。SQLという命令文を実行して作成します。

1. Supabase ダッシュボードの「SQL Editor」を開く

左メニューの「SQL Editor」をクリック。

2. supabase/schema.sql の中身を貼り付けて実行する

ダウンロードしたソースコードの `supabase/schema.sql` をテキストエディタで開き、**全文をコピー**して SQL Editor に貼り付け、「Run」をクリック。

エラーが出なければOKです。

3. 追加のSQLファイルを順番に実行する

`supabase/migrations/` 配下の SQL ファイルを**ファイル名の昇順で全て**実行してください。代表的なファイルとその目的は以下のとおりです。

基本系（必須・name 順）

順番	ファイル名	内容
1	<code>add_ai_logs.sql</code>	AI解析のログテーブル
2	<code>add_candidate_skills.sql</code>	スキルのカテゴリ分けテーブル（14カテゴリの CHECK 制約）
3	<code>add_data_env.sql</code>	本番/デモの環境分けカラム
4	<code>add_project_detail_fields.sql</code>	案件の詳細項目
5	<code>add_projects_updated_by.sql</code>	案件の更新者記録
6	<code>add_updated_by.sql</code>	人材の更新者記録
7	<code>add_email_settings.sql</code>	メール設定（アドレス・ポーリングモード等）
8	<code>add_box_columns.sql</code>	candidates に box_url / box_status を追加
9	<code>add_resume_url.sql</code>	candidates に resume_url / drive_url / desired_rate / from_company を追加
10	<code>add_skill_master.sql</code>	スキルマスタテーブル + match_count RPC
11	<code>seed_skill_master.sql</code>	スキルマスタ初期データ（約 1,600 件）
12	<code>add_attachments_bucket.sql</code>	添付ファイル用 Storage バケット
13	<code>add_find_duplicate_candidates_rpc.sql</code>	重複候補者検索 RPC
14	<code>add_search_rpc.sql</code>	検索用 RPC
15	<code>add_email_polling_cron.sql</code>	5分ごとのメールポーリング cron
16	<code>add_auto_match_cron.sql</code>	毎朝 JST 9:00 の自動マッチ cron
17	<code>add_skill_cleanup_cron.sql</code>	毎日 JST 3:00 のスキルマスタクリーンアップ cron
18	<code>add_enrich_cron.sql</code>	Box 連携の再解析 cron（Box 運用時のみ必要）

順番	ファイル名	内容
19	<code>add_archive_candidates_cron.sql</code>	Phase 4.12 : 7 日アーカイブ cron (毎日 JST 0:00 ・ 人材マップ全期間集計用)。旧 <code>delete-old-candidates</code> を自動 unschedule

タイムスタンプ付き (Phase 4.9 以降の追加・必ずこの順で)

順番	ファイル名	内容
20	<code>20260519090000_add_relevance_keywords.sql</code> ほか	関連性キーワード辞書 + tighten 系 (ファイル名順)
21	<code>20260520121447_fix_skill_master_quality.sql</code>	JP1/Teraterm/Zabbix 等 32 件のスキル追加と alias 修正
22	<code>20260520130000_add_work_phases.sql</code>	構築/テスト/運用 等の工程系 + IBM AIX/DB2/WebSphere/Tivoli + NetApp/EMC/Hitachi 等 18 件追加
23	<code>20260521000000_add_search_scope.sql</code>	<code>search_candidates(p_scope)</code> に <code>tags / body / all</code> の 3 モードを追加
24	<code>20260521210000_add_bigquery_and_cloud_dwh.sql</code>	BigQuery/Redshift/Athena/Synapse/Databricks + Spark/Airflow/Looker Studio/Power BI/Tableau の 10 件追加
25	<code>20260522_add_error_logs.sql</code>	クライアントエラーログテーブル新規作成 (フロントエラー追跡用)
26	<code>20260522_add_fetch_candidates_for_matching.sql</code>	MatchingPage 用 RPC (人材→案件・直近30日順 800件)
27	<code>20260522_add_fetch_candidates_for_project.sql</code>	案件→人材方向の SQL 絞り込み RPC (jsonb skills の <code>&&</code> 演算子問題を回避)
28	<code>20260523_add_process_skills.sql</code>	テスト / 保守開発 / 保守運用 / 調査分析 を methodologies に追加
29	<code>20260523_prefecture_counts_rpc.sql</code>	Phase 4.12 / 人材マップ : 初版 <code>prefecture_counts</code> RPC (後段で上書き)
30	<code>20260523_archive_light_table.sql</code>	Phase 4.12 / 人材マップ : <code>candidates_archive_light</code> テーブル + 期間対応 <code>prefecture_counts(text, text, text)</code> RPC
31	<code>20260523_normalize_prefecture.sql</code>	Phase 4.12 / 人材マップ : <code>normalize_prefecture()</code> 関数 + 最終版 <code>prefecture_counts / candidates_by_prefecture</code> RPC

人材マップ用の追加 SQL (migration 漏れ対応)

20260523_archive_light_table.sql には name / subject カラムが含まれていないが、archive-candidates Edge Function と candidates_by_prefecture RPC は両カラムを参照する。SQL Editor で以下を 1 回だけ実行すること:

```
ALTER TABLE candidates_archive_light
  ADD COLUMN IF NOT EXISTS name text,
  ADD COLUMN IF NOT EXISTS subject text;
```

ファイル名にタイムスタンプ (YYYYMMDDHHMMSS_ プレフィックス) が付いているものはその順で。それ以外は add_*.sql の名前順で OK です。*_cron.sql 系は YOUR_PROJECT_REF と YOUR_SERVICE_ROLE_KEY を実際の値に書き換えてから実行してください。

⚠️ 重要: schema.sql の candidate_skills.check_category は 14 カテゴリへ更新済みですが、add_candidate_skills.sql 等のマイグレーションも必ず流してください。⚠️ 重要: fetch_candidates_for_matching / fetch_candidates_for_project RPC は MatchingPage の動作に必須です。これらを流さないとマッチング画面が空になります。⚠️ 重要: 人材マップ機能を使う場合は prefecture_counts / candidates_by_prefecture / normalize_prefecture の 3 つを流したうえで、上記の ALTER TABLE も必須です。

完了チェック

- ☐ Supabaseのプロジェクトを作成した
- ☐ Project URL・anon キー・service_role キーをメモした
- ☐ schema.sql を実行した
- ☐ supabase/migrations/ 配下を全てファイル名順に実行した
- ☐ *_cron.sql 系は YOUR_PROJECT_REF / YOUR_SERVICE_ROLE_KEY を置換した上で実行した

第3章 AIの設定 (APIキーの取得)

重要: 2026-05-19 のコミット 139a4f2 でメール解析 (inbound-email) から AI 利用が完全に除去され、コミット a4dc3b4 でデッドコードも削除されました。2026-05-22 のコミット b35df40 で新しい match-batch Edge Function が導入され、マッチング処理は「ルールベース事前フィルタ + バッチ AI 採点」方式になりました。現在 AI を使うのは マッチング処理 (match-batch / match-score / auto-match) と poll-email メール種別分類 (任意・既定 OFF) だけです。

AI	主な用途	必須度
Cerebras	match-batch / match-score の 1 段目 (軽量・実質無制限)	推奨 (高速化に寄与)
Groq	match-batch / match-score の 2 段目 (高精度モデル llama-3.3-70b-versatile)	◎ 必須
Gemini	match-batch / match-score の 最終フォールバック・poll-email 種別分類	◎ 必須

取得したAPIキーは第4章と第6章でまとめて登録します。ここではメモするだけでOKです。3 段すべて失敗してもルールスコアで全代替されるため、システム自体は止まりませんが、AI 採点なしでは品質が落ちるので必ず Groq と Gemini は登録してください。

3-1. Gemini APIキーを取得する (必須)

1. <https://aistudio.google.com> をブラウザで開く

Googleアカウントでログインします。

2. 「Get API key」をクリック

3. 「Create API key」をクリックしてAPIキーを発行する

表示されたキー（`AIza...` のような文字列）をメモしてください。

Gemini はプリペイド制（従量課金）です。無料枠はありません。クレジット切れの場合は `auto-match` のスコア計算が失敗します。

3-2. Groq APIキーを取得する（必須）

Groq は `match-score`（手動マッチング）の2段階目モデル（`llama-3.3-70b-versatile`）として使います。無料枠は 500K tokens/日（JST 9:00 リセット）で、マッチング用なら約 300 ペア/日に相当します。

1. <https://console.groq.com> をブラウザで開く

アカウントを作成（または Google アカウントでログイン）します。

2. 「API Keys」→「Create API Key」でキーを発行する

表示されたキー（`gsk_...` のような文字列）をメモしてください。

3-3. Cerebras APIキーを取得する（推奨）

Cerebras は `match-score` の1段階目（軽量モデル `llama3.1-8b`）として使います。無料枠が非常に大きい実質無制限で、ここで成功すれば Groq を消費しません。

1. <https://cloud.cerebras.ai> をブラウザで開く

アカウントを作成（または Google アカウントでログイン）します。

2. 「API Keys」→「Generate API Key」でキーを発行する

表示されたキーをメモしてください。

完了チェック

- ☐ Google AI Studio で Gemini APIキーを取得し、メモした（必須）
- ☐ Groq で APIキーを取得し、メモした（必須）
- ☐ Cerebras で APIキーを取得し、メモした（推奨）

第4章 サーバー機能のデプロイ（Edge Functions）

「Edge Functions」とは、Supabase のサーバー上で動くプログラムです。
このシステムでは以下の Edge Functions をデプロイします。

Edge Function	役割
<code>inbound-email</code>	メール解析（AI 不使用・regex + DB 照合のみ）

Edge Function	役割
<code>poll-email</code>	Outlook のメール取得 (5 分ごと cron)
<code>auto-match</code>	毎朝 JST 9:00 の自動マッチング (<code>match-batch</code> を内部呼び出し)
<code>match-batch</code>	バッチ AI 採点 (ルール事前フィルタ + topN を 1 コール採点・Phase 4.10 新規)
<code>match-score</code>	UI から呼ばれる単発スコア計算 (duplicate 検出付き)
<code>microsoft-oauth</code>	Microsoft アカウント連携 (OAuth コールバック)
<code>enrich-candidate</code>	Box 連携・再解析 (Box 運用時のみ)
<code>skill-master-cleanup</code>	<code>skill_master</code> の毎日クリーンアップ

4-1. Supabase CLIでログインする

```
npx supabase login
```

ブラウザが自動で開くのでログインしてください。

4-2. このプロジェクトに接続する

「Reference ID」(プロジェクトID) を Supabase ダッシュボードの「Settings」→「General」→「Reference ID」で確認してメモしてください。

```
cd akinavi-hr-ai
npx supabase link --project-ref (Reference IDを貼り付け)
```

4-3. Edge Functions をデプロイする

```
npx supabase functions deploy inbound-email
npx supabase functions deploy poll-email
npx supabase functions deploy auto-match
npx supabase functions deploy match-batch # Phase 4.10 新規
npx supabase functions deploy match-score
npx supabase functions deploy archive-candidates # Phase 4.12 新規 (人材マップの 7
日アーカイブ用)
npx supabase functions deploy microsoft-oauth
npx supabase functions deploy enrich-candidate
npx supabase functions deploy skill-master-cleanup
```

それぞれ「Deployed」と表示されればOKです。

デプロイ前に型検査したい場合は `npm run check:edge <function>` を使うと `deno check` で TS2304 (未定義変数) を検知し、エラーがあればデプロイを中止できます。`npm run deploy:edge <function>` で「型検査 + デプロイ」をまとめて実行できます。引数を省略すると `inbound-email` を対象とします。

4-4. Secrets (機密情報) を登録する

「Secrets」とは、サーバー上のプログラムが使うAPIキーやパスワードを安全に保管する場所です。
Supabase ダッシュボード → 「Edge Functions」 → 「Secrets」 → 「Add new secret」 から登録します。

今すぐ登録するもの

Secret名	値	必須
GEMINI_API_KEY	第3章でメモしたGemini APIキー（ <code>auto-match</code> 等で使用）	◎
GROQ_API_KEY	第3章でメモしたGroq APIキー（ <code>match-score</code> で使用）	◎
CEREBRAS_API_KEY	第3章でメモしたCerebras APIキー（ <code>match-score</code> 1 段目）	推奨
INBOUND_CALL_KEY	第2章でメモした <code>service_role</code> キー	◎

`SUPABASE_URL` と `SUPABASE_SERVICE_ROLE_KEY` は Supabase が自動で設定するため、手動登録は不要です。もしエラーが出る場合は手動で追加してください。

`inbound-email` は AI を使わなくなったため、上記の API キーがなくてもメール解析自体は動きます。ただしマッチング処理が動かないと意味がないので必ず設定してください。

第5章で追加登録するもの（今はスキップ）

Secret名	用途
GRAPH_CLIENT_ID	第5章で取得
GRAPH_CLIENT_SECRET	第5章で取得
GRAPH_REFRESH_TOKEN_HUMAN	第5章で取得
GRAPH_REFRESH_TOKEN_PROJECT	第5章で取得
GRAPH_REFRESH_TOKEN_HUMAN_DEV	第5章で取得
GRAPH_REFRESH_TOKEN_PROJECT_DEV	第5章で取得

Box 連携を使う場合のみ

Secret名	用途
GOOGLE_SERVICE_ACCOUNT_JSON	Box → Drive 移送用キュー（スプレッドシート）アクセス
BOX_SPREADSHEET_ID	キュー用スプレッドシート ID

完了チェック

- ☐ `supabase login` が完了した
- ☐ `supabase link` でプロジェクトに接続した
- ☐ 全Edge Functions（8つ）をデプロイした
- ☐ `GEMINI_API_KEY`・`GROQ_API_KEY`・`CEREBRAS_API_KEY`・`INBOUND_CALL_KEY` を Secrets に登録した

第5章 メール自動取り込みの設定

この章では、第0章で作成した4つのOutlookアカウントと、このシステムを連携させます。

「5分ごとに未読メールを自動で取得・解析・保存する」という仕組みを作ります。

設定には3つのステップがあります。

- ① Azureにアプリを登録する（接続許可の設定）
- ② 各Outlookアカウントのリフレッシュトークンを取得する
- ③ Supabaseにスケジューラを登録する

5-1. Azureにアプリを登録する

「Azure」は Microsoft のクラウドサービスです。ここでアプリを登録することで、このシステムがOutlookにアクセスする許可を得ます。**Microsoftアカウントがあれば無料で使えます。**

1. <https://portal.azure.com> をブラウザで開く

Microsoftアカウント（Outlookアカウントのどれかでも可）でログインします。

2. 「Microsoft Entra ID」を検索して開く

上部の検索バーに「Microsoft Entra ID」と入力して選択します。

3. 「アプリの登録」→「新規登録」をクリック

4. 以下を入力して「登録」をクリック

項目	入力内容
名前	任意（例: akinavi-mail-reader ）
サポートされるアカウントの種類	「 個人用 Microsoft アカウントのみ 」を選択
リダイレクト URI	「Web」を選び、 http://localhost と入力

5. クライアントIDをメモする

登録完了画面に表示される「**アプリケーション（クライアント）ID**」をメモします。

6. クライアントシークレットを作成する

左メニュー「証明書とシークレット」→「新しいクライアントシークレット」→説明を入力して「追加」。

表示された「**値**」をメモします。

この画面を閉じると二度と確認できないので必ずメモしてください。

7. APIの権限を追加する

左メニュー「APIのアクセス許可」→「アクセス許可の追加」→「Microsoft Graph」→「委任されたアクセス許可」で以下を検索して追加:

- [Mail.Read](#)
- [Mail.ReadWrite](#)

追加後、「(テナント名) に管理者の同意を与えます」のボタンが表示される場合はクリックしてください。

この時点でメモしたもの:

- クライアントID → Supabase Secrets の **GRAPH_CLIENT_ID** に登録
- クライアントシークレットの値 → Supabase Secrets の **GRAPH_CLIENT_SECRET** に登録

今すぐ Supabase ダッシュボード → 「Edge Functions」 → 「Secrets」 に登録してください。

5-2. 各Outlookアカウントのリフレッシュトークンを取得する

「リフレッシュトークン」とは、このシステムがOutlookにアクセスし続けるための認証情報です。
4つのアカウント分を取得します。

1. 以下のURLをブラウザで開く

(**クライアントID**) の部分を 5-1 でメモしたクライアントIDに置き換えてください。

```
https://login.microsoftonline.com/consumers/oauth2/v2.0/authorize?client_id= (クライアントID)
&response_type=code&redirect_uri=http://localhost&scope=offline_access%20Mail.Read%20Mail.ReadWrite
```

2. 1つ目のOutlookアカウント（人材用・本番）でログインする

ログインするとブラウザが **http://localhost/?code= (長い文字列)** というURLに遷移します。
(ページは表示されませんが問題ありません)

アドレスバーの **code=** 以降の文字列を全部コピーしてメモします。

3. コードをリフレッシュトークンに変換する

ターミナルで以下を実行します (**(...)** 部分を実際の値に置き換え) :

```
curl -X POST https://login.microsoftonline.com/consumers/oauth2/v2.0/token \
-d "client_id= (クライアントID) " \
-d "client_secret= (クライアントシークレット) " \
-d "redirect_uri=http://localhost" \
-d "grant_type=authorization_code" \
-d "code= (コピーしたcode) " \
-d "scope=offline_access Mail.Read Mail.ReadWrite"
```

返ってきたJSON の **"refresh_token"** の値をメモします。これが **リフレッシュトークン**です。

4. 残り3アカウント分も繰り返す

同じ手順を残り3つのアカウントで繰り返します。ブラウザで別のアカウントにログインするときは、**一度ログアウトしてから**次のアカウントでログインしてください。

アカウント	Secretキー名
人材用・本番	GRAPH_REFRESH_TOKEN_HUMAN
案件用・本番	GRAPH_REFRESH_TOKEN_PROJECT
人材用・デモ	GRAPH_REFRESH_TOKEN_HUMAN_DEV

アカウント	Secretキー名
案件用・デモ	GRAPH_REFRESH_TOKEN_PROJECT_DEV

5. Supabase Secrets に登録する

Supabase ダッシュボード → 「Edge Functions」 → 「Secrets」 で、以下を登録します:

Secret名	値
GRAPH_REFRESH_TOKEN_HUMAN	人材用・本番のリフレッシュトークン
GRAPH_REFRESH_TOKEN_PROJECT	案件用・本番のリフレッシュトークン
GRAPH_REFRESH_TOKEN_HUMAN_DEV	人材用・デモのリフレッシュトークン
GRAPH_REFRESH_TOKEN_PROJECT_DEV	案件用・デモのリフレッシュトークン

5-3. 拡張機能を有効にする

Supabase のスケジューラ機能 (pg_cron) と HTTP通信機能 (pg_net) を有効にします。

Supabase ダッシュボード → 「Database」 → 「Extensions」 を開き、以下を検索してONにします:

- pg_cron
- pg_net

5-4. 5分ごとのスケジュールを登録する

supabase/migrations/add_email_polling_cron.sql をテキストエディタで開き、以下の2箇所を書き換えます:

書き換え前	書き換え後
YOUR_PROJECT_REF	Supabase の Reference ID (第4章でメモしたもの)
YOUR_SERVICE_ROLE_KEY	Supabase の service_role キー (第2章でメモしたもの)

書き換えたら、Supabase の SQL Editor に全文貼り付けて「Run」をクリック。

最後の SELECT 結果に poll-email-every-5-minutes というジョブが表示されればOKです。

完了チェック

- ☐ Azureにアプリを登録し、クライアントID・シークレットをメモした
- ☐ Mail.Read / Mail.ReadWrite の権限を追加した
- ☐ GRAPH_CLIENT_ID / GRAPH_CLIENT_SECRET を Secrets に登録した
- ☐ 4つのアカウントのリフレッシュトークンを取得した
- ☐ 4つのリフレッシュトークンを Secrets に登録した
- ☐ pg_cron・pg_net を有効にした
- ☐ add_email_polling_cron.sql を書き換えて実行した

第6章 本番サイトの公開 (Vercel)

「Vercel」とは、作ったWebサイトをインターネット上に公開するためのサービスです。無料で使えます。
ここで環境変数(設定値)をまとめて登録してデプロイすると、本番サイトが完成します。

6-1. Vercelにサインアップ・プロジェクトをインポートする

1. <https://vercel.com>を開き、GitHubアカウントでサインアップ

2. 「Add New Project」 → 「Import Git Repository」でこのリポジトリ ([akinavi-hr-ai](#)) を選択

3. 「Environment Variables」に以下をすべて入力する

「Deploy」ボタンを押す前に、以下の環境変数を全部登録してください。

変数名	値	メモした場所
VITE_SUPABASE_URL	Supabase の Project URL	第2章
VITE_SUPABASE_ANON_KEY	Supabase の anon キー	第2章
VITE_GEMINI_API_KEY	Gemini の APIキー	第3章
VITE_AI_PROVIDER	gemini (そのまま入力)	—
VITE_DEMO_KEY	任意の文字列 (例: demo2024)	—

[VITE_DEMO_KEY](#) はデモ環境の解除キーです。自分で決めた文字列を設定してください。
詳細は [docs/DataEnv_Demo_Prod.md](#) を参照。

4. 「Deploy」をクリックする

デプロイが完了すると [https://\(プロジェクト名\).vercel.app](#) のような URLが発行されます。
このURLが本番サイトのアドレスになります。

以降は [main](#) ブランチに変更を push するたびに**自動で再デプロイ**されます。

6-2. 環境変数を後から変更する場合

Vercel ダッシュボード → プロジェクトを選択 → 「Settings」 → 「Environment Variables」から変更できます。
変更後は「Deployments」 → 「Redeploy」で再デプロイが必要です。

完了チェック

- ☐ Vercel にプロジェクトをインポートした
- ☐ 5つの環境変数をすべて設定した
- ☐ デプロイが成功してURLが発行された
- ☐ 発行されたURLでサイトが表示された

最終確認チェックリスト

全章が完了したら、以下を本番URLで確認してください。

- ☐ 本番URLでブラウザにエラーなく画面が表示される
- ☐ 人材登録タブでテキストを貼り付けて「登録」が動く (Phase 4.11 で「AI で登録」は廃止・登録ボタンに統一)
- ☐ マッチング結果タブでスコアが表示される
- ☐ マッチング詳細パネルに案件サマリーが表示される
- ☐ 専用のOutlookアドレスにメールを送って5分以内に人材/案件が登録される

- ☐ **人材マップタブ**で日本地図が表示され、登録済み人材の都道府県が色付けされる
- ☐ **人材マップ**で都道府県をクリックすると下部にメール一覧が出る
- ☐ **人材マップ**でスキル名（例: **Java**）を入力するとオートコンプリート候補が出る
- ☐ **人材マップ**の「全期間」モードに切り替えてもエラーが出ない（**candidates_archive_light** の **name / subject** カラムが必要）
- ☐ **[station_unmapped]** ログが Supabase Functions Logs に流れていないか確認（月次レビュー）

うまくいかないときは

画面が真っ白になる・エラーが出る

ブラウザの開発者ツールを開いて（F12 キー）、「Console」タブにエラーメッセージが出ていないか確認する。

よくある原因:

- Vercel の環境変数に誤りがある（スペースや余分な文字が入っていないか確認）
- 環境変数を設定後に再デプロイしていない

AIが解析されない（登録できない）

- Vercel の **VITE_GEMINI_API_KEY** が正しく設定されているか確認
- Supabase の「Edge Functions」→「Logs」→「inbound-email」でエラーが出ていないか確認

メールが自動取り込みされない

Supabase の「Edge Functions」→「Logs」→「poll-email」のログを確認する。

よくある原因:

- Graph API の Secrets が1つでも登録されていない
- リフレッシュトークンが間違っている（取得後に有効期限が切れた場合は再取得が必要）
- **pg_cron** または **pg_net** が有効になっていない
- **add_email_polling_cron.sql** の **YOUR_PROJECT_REF / YOUR_SERVICE_ROLE_KEY** を書き換えずに実行してしまった

マイグレーションでエラーが出る

- **schema.sql** を先に実行したか確認
- 同じSQLを二重実行していないか確認（多くのSQLは **IF NOT EXISTS** で二重実行に対応しているが念のため）

参考ドキュメント

ファイル	内容
README.md	システム全体の概要・技術スタック
docs/Sales_Manual.md	営業担当者向けの操作マニュアル
docs/DataEnv_Demo_Prod.md	本番・デモ環境の切替方法
docs/Heatmap.md	人材マップ（ヒートマップ）機能の詳細仕様
docs/matching_candidate_selection.md	マッチング選定ロジック
docs/ai_fallback_flow.md	AI フォールバックフロー詳細